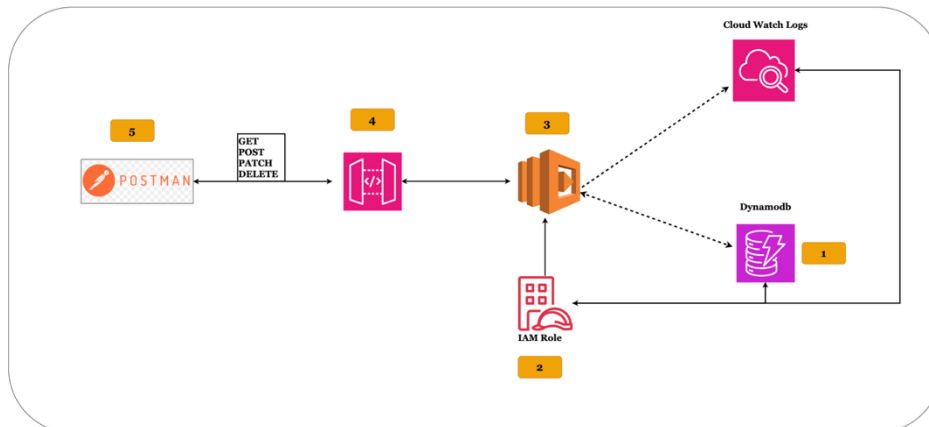
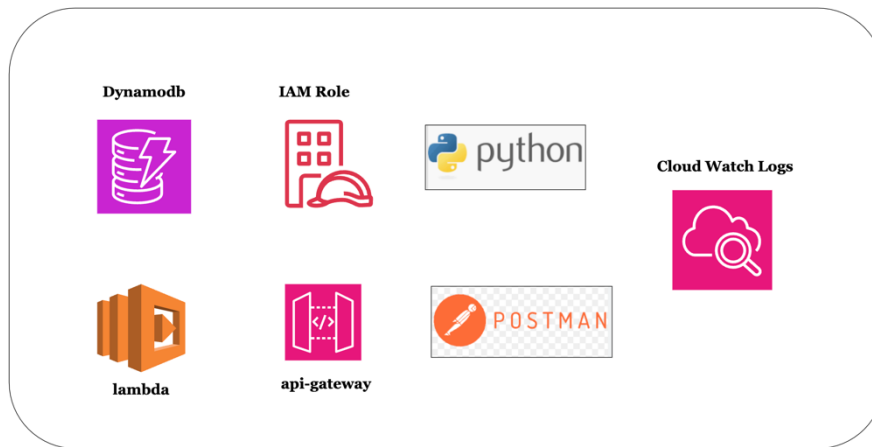


API GATEWAY --> LAMBDA --> DYNAMODB



1.Creating dynamodb:

Table Name: employee-details

Create table

Table details [Info](#)

DynamoDB is a schemaless database that requires only a table name and a primary key when you create the table.

Table name

This will be used to identify your table.

employee-details

Between 3 and 255 characters, containing only letters, numbers, underscores (_), hyphens (-), and periods (.).

Partition key

The partition key is part of the table's primary key. It is a hash value that is used to retrieve items from your table and allocate data across

empid

Strir

1 to 255 characters and case sensitive.

Sort key - optional

You can use a sort key as the second part of a table's primary key. The sort key allows you to sort or search among all items sharing the s

Enter the sort key name

Strir

1 to 255 characters and case sensitive.

Table settings

Default settings

The fastest way to create your table. You can modify most of these settings after your table has been created. To modify these settings now, choose "Customize settings".

Customize settings

Use these advanced features to make DynamoDB work better for your needs.

Default table settings

These are the default settings for your new table. You can change some of these settings after creating the table.

Setting	Value	Editable after creation
Table class	DynamoDB Standard	Yes
Capacity mode	On-demand	Yes
Maximum read capacity units	-	Yes
Maximum write capacity units	-	Yes
Local secondary indexes	-	No
Global secondary indexes	-	Yes
Encryption key management	AWS owned key	Yes
Deletion protection	Off	Yes
Resource-based policy	Not active	Yes

Tags

Tags are pairs of keys and optional values, that you can assign to AWS resources. You can use tags to control access to your resources or track your AWS spending.

No tags are associated with the resource.

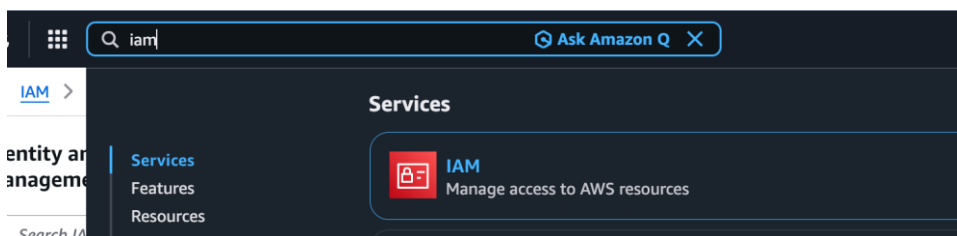
[Add new tag](#)

You can add 50 more tags.

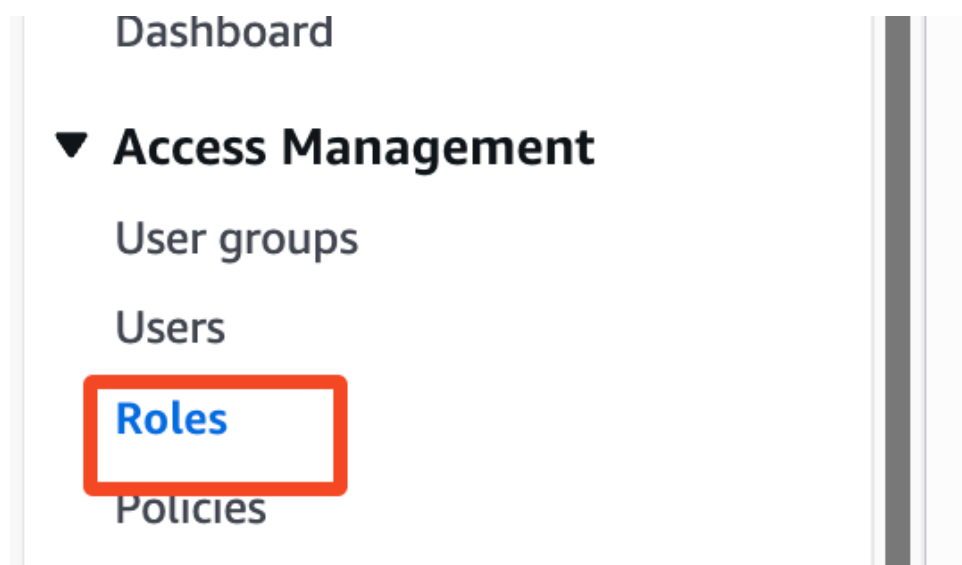
[Cancel](#)

[Create table](#)

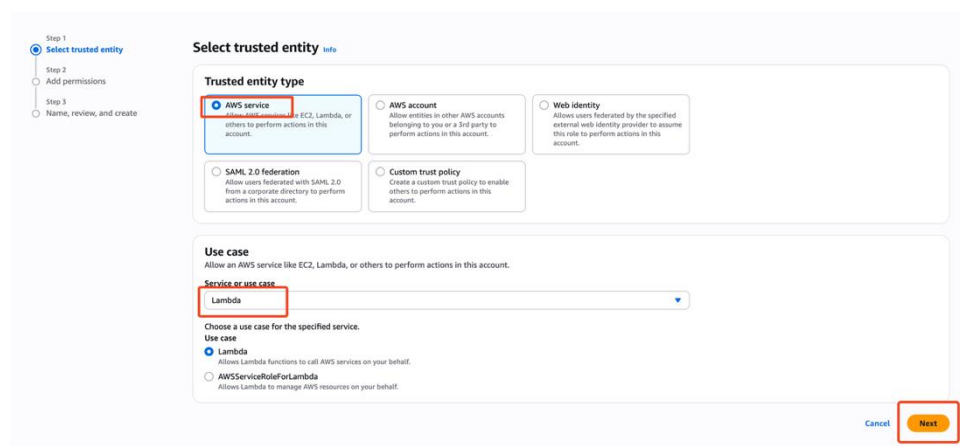
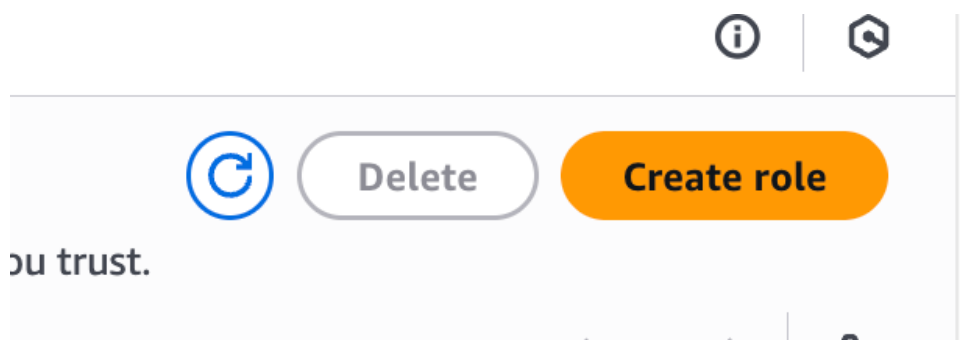
2.IAM Role - LAMBDA



Select the below option role



Click on create role



Policy name

AmazonDynamoDBFullAccess

AmazonDynamoDBFullAccess_v2

CloudWatchLogsFullAccess

CloudWatchLogsReadOnlyAccess

Step 1

Select trusted entity

Step 2

Add permissions

Step 3

Name, review, and create

Name, review, and create

Role details

Role name

Enter a meaningful name to identify this role.

AWS-PROJECT-ROLE

Maximum 64 characters. Use alphanumeric and '+=, @, _' characters.

Description

Add a short explanation for this role.

Allows Lambda functions to call AWS services on your behalf.

Step 2: Add permissions

Edit

Permissions policy summary

Policy name	Type	Attached as
AmazonDynamoDBFullAccess	AWS managed	Permissions policy
AmazonDynamoDBFullAccess_v2	AWS managed	Permissions policy
CloudWatchLogsFullAccess	AWS managed	Permissions policy

Step 3: Add tags

Add tags - optional

Tags are key-value pairs that you can add to AWS resources to help identify, organize, or search for resources.

No tags associated with the resource.

Add new tag

You can add up to 50 more tags.

Cancel

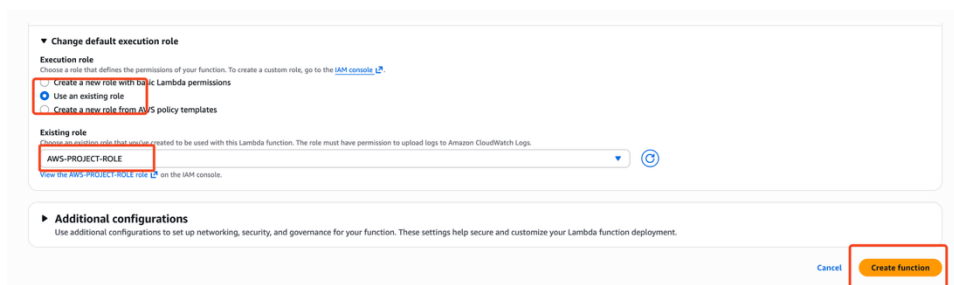
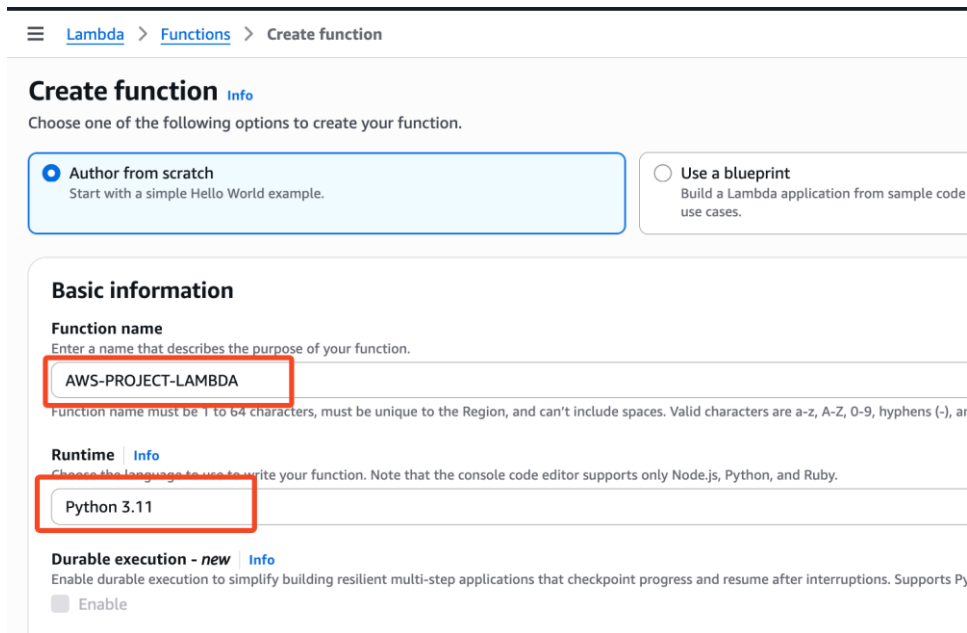
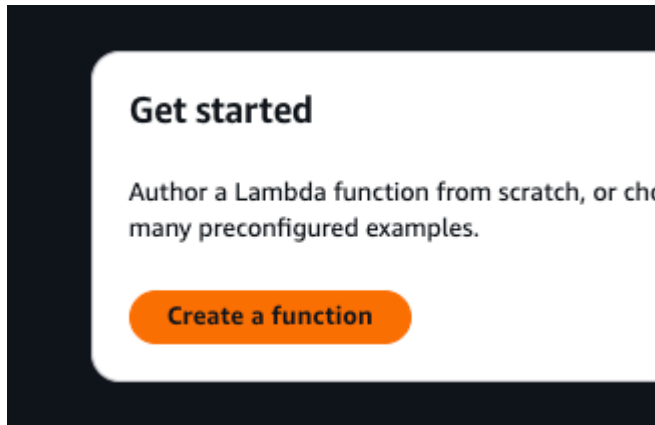
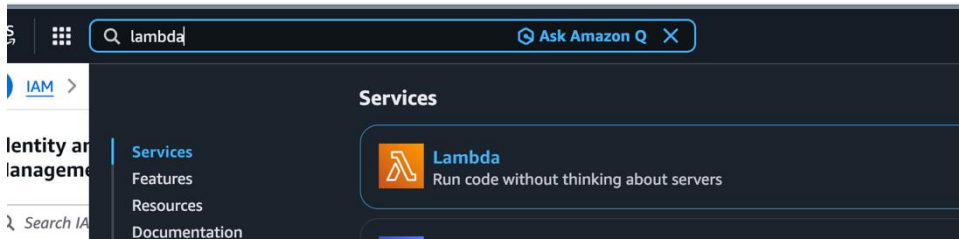
Previous

Create role

3.Creating LAMBDA

Create a lambda with role created above

BUSINESS DOCUMENT This document is intended for business use and should be distributed to intended recipients only.



Python Code:

```
import json
import boto3
import logging
from botocore.exceptions import ClientError

# -----
# Logging Configuration
# -----
logger = logging.getLogger()
logger.setLevel(logging.INFO)

# -----
# DynamoDB Setup
# -----
dynamodb = boto3.resource("dynamodb")
TABLE_NAME = "employee-details"
table = dynamodb.Table(TABLE_NAME)

# -----
# Common Response Formatter
# -----
def process_response(status_code, body):
    logger.info(f"Returning response | status={status_code} | body={body}")

    return {
        "statusCode": status_code,
        "headers": {
            "Content-Type": "application/json"
        },
        "body": json.dumps(body, default=str)
    }

# -----
# READ Item
# -----
def read_item(empid):
    logger.info(f"Reading employee | empid={empid}")

    try:
        result = table.get_item(Key={"empid": empid})
        logger.debug(f"DynamoDB raw response: {result}")

        if "Item" in result:
            logger.info(f"Employee found | empid={empid}")
            return process_response(200, result["Item"])
        else:
            logger.warning(f"Employee NOT found | empid={empid}")
            return process_response(404, f"No employee found with empid {empid}")

    except ClientError as e:
        logger.error(f"DynamoDB read failed | empid={empid} | error={str(e)}")
```

```

        return process_response(500, str(e))

# -----
# CREATE Item
# -----
def create_item(payload):
    logger.info(f"Creating employee | payload={payload}")

    try:
        table.put_item(Item=payload)
        return process_response(201, "Employee created successfully")

    except ClientError as e:
        logger.error(f"DynamoDB create failed | error={str(e)}")
        return process_response(500, str(e))

# -----
# UPDATE Item
# -----
def update_item(empid, update_data):
    logger.info(f"Updating employee | empid={empid} | update_data={update_data}")

    try:
        update_expression = "SET " + ", ".join(
            [f"{k} = :{k}" for k in update_data.keys()]
        )

        expression_values = {f":{k}": v for k, v in update_data.items()}

        response = table.update_item(
            Key={"empid": empid},
            UpdateExpression=update_expression,
            ExpressionAttributeValues=expression_values,
            ReturnValues="UPDATED_NEW"
        )

        logger.info(f"Update successful | response={response}")
        return process_response(200, "Employee updated successfully")

    except ClientError as e:
        logger.error(f"DynamoDB update failed | empid={empid} | error={str(e)}")
        return process_response(500, str(e))

# -----
# DELETE Item
# -----
def delete_item(empid):
    logger.info(f"Deleting employee | empid={empid}")

    try:
        table.delete_item(Key={"empid": empid})

```

```

        return process_response(200, f"Employee {empid} deleted successfully")

except ClientError as e:
    logger.error(f"DynamoDB delete failed | empid={empid} | error={str(e)}")
    return process_response(500, str(e))

# -----
# Lambda Entry Point
# -----
def lambda_handler(event, context):

    request_id = context.aws_request_id
    logger.info(f"Lambda invoked | request_id={request_id}")
    logger.info(f"Incoming event: {json.dumps(event)}")

    try:
        path = event.get("path")
        method = event.get("httpMethod")

        body = json.loads(event["body"]) if event.get("body") else {}
        query_params = event.get("queryStringParameters") or {}
        empid = query_params.get("empid")

        logger.info(f"Routing request | path={path} | method={method}")

        # ----- READ -----
        if path == "/read" and method == "GET":
            if not empid:
                logger.warning("Missing empid in query params")
                return process_response(400, "empid query parameter is required")

            return read_item(empid)

        # ----- CREATE -----
        elif path == "/create" and method == "POST":
            if "empid" not in body:
                logger.warning("empid missing in request body")
                return process_response(400, "empid is required in request body")

            return create_item(body)

        # ----- UPDATE -----
        elif path == "/update" and method == "PUT":
            if not empid:
                logger.warning("Missing empid for update")
                return process_response(400, "empid query parameter is required")

            return update_item(empid, body)

        # ----- DELETE -----
        elif path == "/delete" and method == "DELETE":
            if not empid:
                logger.warning("Missing empid for delete")

```

```
return process_response(400, "empid query parameter is required")
```

```
return delete_item(empid)
```

else:

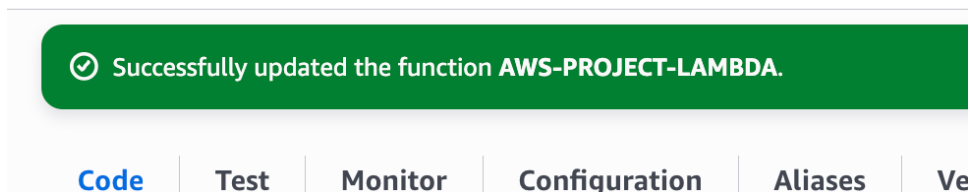
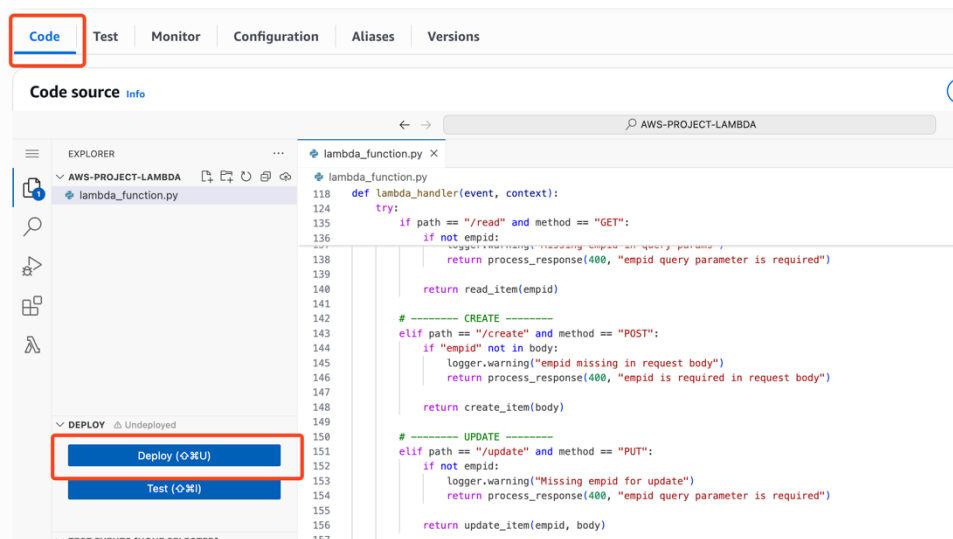
```
logger.warning(f"Invalid route | path={path} | method={method}")
```

```
return process_response(404, "Route not found")
```

except Exception as e:

```
logger.exception("Unhandled Lambda exception")
```

```
return process_response(500, str(e))
```



4.API Gateway

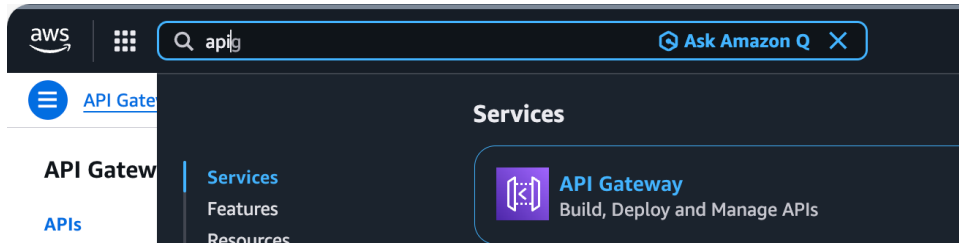
Let's create the API gateway with below - CRUD

/read --> GET

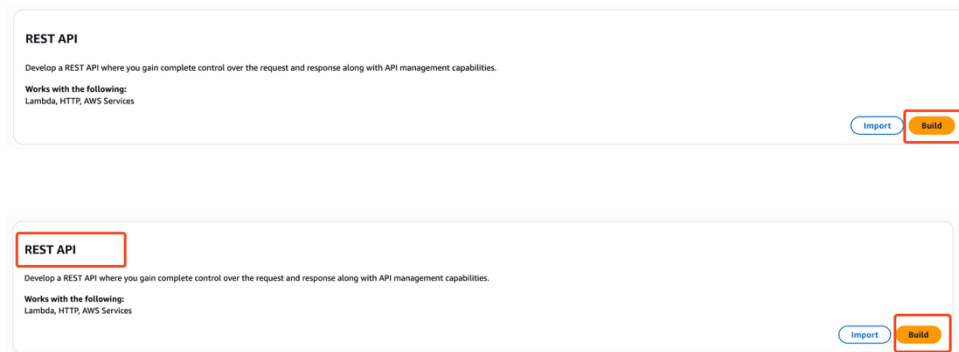
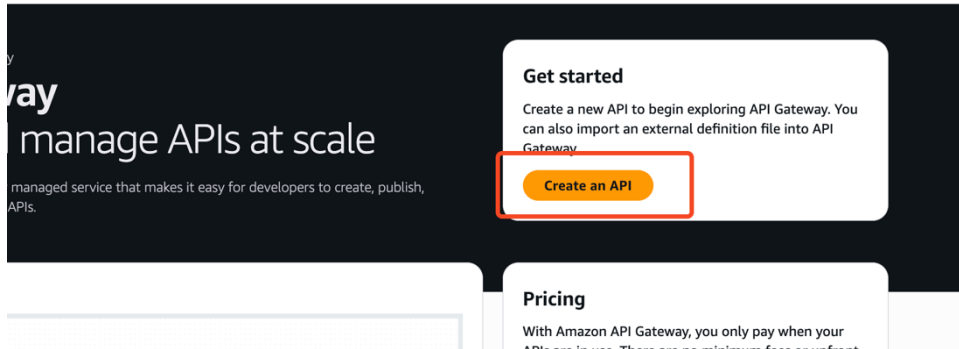
/create --> POST

/update --> PUT

/delete --> DELETE



Creating API Gateway



Create REST API [Info](#)

API details



New API

Create a new REST API.



Import API

Import an API from an OpenAPI definition.

API name

employee-management

Description - optional

employee-management

IP address type [Info](#)
Select the type of IP addresses that can invoke the default endpoint for your API.

☒ **IPv4**
Supports only edge-optimized and Regional API endpoint types.

☐ **Dualstack**
Supports all API endpoint types.

[Cancel](#)

Create API

Creating resources:

Read, Create, Update, Delete

API Gateway

APIs
Custom domain names
Domain name access associations
VPC links
AgentCore targets [New](#)

API: employee-management

[Resources](#)

[Stages](#)

[Authorizers](#)

[Gateway responses](#)

[Models](#)

[Resource policy](#)

[Documentation](#)

[Dashboard](#)

Successfully created REST API 'employee-management (pl4pfeimz1)'.

Resources

Create resource

Resource details

Path

/

R

7

Methods (0)

Method type



Integration type

No meth

No methods

Resource details

☒ Proxy resource [Info](#)

Proxy resources handle requests to all sub-resources. To create a proxy resource use a path parameter that ends with a plus sign, for example {proxy+}.

Resource path

/

Resource name

create

☐ CORS (Cross Origin Resource Sharing) [Info](#)

Create an OPTIONS method that allows all origins, all methods, and several common headers.

[Cancel](#)

Create resource

Create resource

Resource details

☒ Proxy resource [info](#)
Proxy resources handle requests to all sub-resources. To create a proxy resource use a path parameter that ends with a plus sign, for example {proxy+}.

Resource path

Resource name

☐ CORS (Cross Origin Resource Sharing) [info](#)
Create an OPTIONS method that allows all origins, all methods, and several common headers.

[Cancel](#) [Create resource](#)

Create resource

Resource details

☒ Proxy resource [info](#)
Proxy resources handle requests to all sub-resources. To create a proxy resource use a path parameter that ends with a plus sign, for example {proxy+}.

Resource path

Resource name

☐ CORS (Cross Origin Resource Sharing) [info](#)
Create an OPTIONS method that allows all origins, all methods, and several common headers.

[Cancel](#) [Create resource](#)

Create resource

Resource details

☒ Proxy resource [info](#)
Proxy resources handle requests to all sub-resources. To create a proxy resource use a path parameter that ends with a plus sign, for example {proxy+}.

Resource path

Resource name

☐ CORS (Cross Origin Resource Sharing) [info](#)
Create an OPTIONS method that allows all origins, all methods, and several common headers.

[Cancel](#) [Create resource](#)

API Gateway

- APIs
- Custom domain names
- Domain name access associations
- VPC links
- AgentCore targets [New](#)

API: employe-management

- Resources**
- Stages
- Authorizers
- Gateway responses
- Models
- Resource policy
- Documentation
- Dashboard

Resources

[Create resource](#)

- /
- /create
- /delete**
- /read
- /update

Resource details

Path
/delete

Methods (0)

Method type

✓ Successfully created resource '/delete'

Let's create methods for the each resource

- /read --> GET
- /create --> POST
- /update --> PUT
- /delete --> DELETE

Read:

Resources

Create resource

/

/create

/delete

/read

/update

Resource details

Path: /read

Resource ID: pwkzje

Delete Update documentation Enable CORS

Methods (0)

Delete Create method

Method type Integration type Authorization API key

No methods

No methods defined.

Create method

Method details

Method type: GET

Integration type

☒ **Lambda function**
Integrate your API with a Lambda function.

☐ **HTTP**
Integrate with an existing HTTP endpoint.

☐ **AWS service**
Integrate with an AWS Service.

☐ **VPC link**
Integrate with a resource that isn't accessible over the public i

☒ **Lambda proxy integration**
Send the request to your Lambda function as a structured event.

Response transfer mode Info

☒ **Buffered**
Wait to receive the complete response before beginning transmission.

☐ **Stream**
Send portions of the response without waiting for the complete response.

☐ **Stream**
Send portions of the response without waiting for the complete response.

Lambda function
Provide the Lambda function name or alias. You can also provide an ARN from another account.

us-east-1

☒ **Grant API Gateway permission to invoke your Lambda function**
When you save your changes, API Gateway updates your Lambda function's resource-based policy to allow this API to invoke it.

Integration timeout Info
By default, you can enter an integration timeout of 50 - 29,000 milliseconds. You can use Service Quotas to raise the integration timeout to greater than 29,000 ms.

29000

Method request settings

URL query string parameters

HTTP request headers

Request body

Cancel Create method

CREATE - POST:

/

/create

/delete

/read

GET

/update

Path

/create

Resource ID

fdhila

Methods (0)

Delete

Create method

Method type

Integration type

Authorization

API key

No methods

No methods defined.

Create method

Method details

Method type

POST

Integration type

☒ Lambda function

Integrate your API with a Lambda function.



☐ HTTP

Integrate with an existing HTTP endpoint.



☐ AWS service

Integrate with an AWS Service.



☐ VPC link

Integrate with a resource that isn't accessible



☒ Lambda proxy integration

Send the request to your Lambda function as a structured event.

Response transfer mode

[Info](#)

☒ Buffered

Wait to receive the complete response before beginning transmission.

Lambda function

Provide the Lambda function name or alias. You can also provide an ARN from another account.

us-east-1



Q. arnaws:lambda:us-east-1:572941607001:function:AWS-PROJECT-LJ X

☒ Grant API Gateway permission to invoke your Lambda function

When you save your changes, API Gateway updates your Lambda function's resource-based policy to allow this API to invoke it.

Integration timeout

[Info](#)

By default, you can enter an integration timeout of 50 - 29,000 milliseconds. You can use Service Quotas to raise the integration timeout to greater than 29,000 ms

29000

Method request settings

URL query string parameters

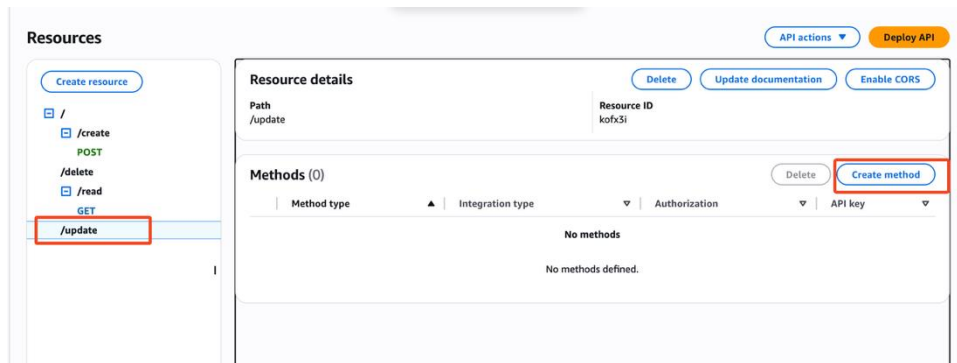
HTTP request headers

Request body

Cancel

Create method

UPDATE - PUT



Create method

Method details

Method type

PUT

Integration type

☒ **Lambda function**
Integrate your API with a Lambda function.

☐ **HTTP**
Integrate with an existing HTTP endpoint.

☐ **AWS service**
Integrate with an AWS Service.

☐ **VPC link**
Integrate with a resource that isn't accessible c

☒ **Lambda proxy integration**
Send the request to your Lambda function as a structured event.

Lambda function

us-east-1

arn:aws:lambda:us-east-1:1672941607001:function:AWS-PROJECT-1

☒ Grant API Gateway permission to invoke your Lambda function
When you save your changes, API Gateway updates your Lambda function's resource-based policy to allow this API to invoke it.

Integration timeout Info
By default, you can enter an integration timeout of 50 - 29,000 milliseconds. You can use Service Quotas to raise the integration timeout to greater than 29,000 ms.

29000

► Method request settings

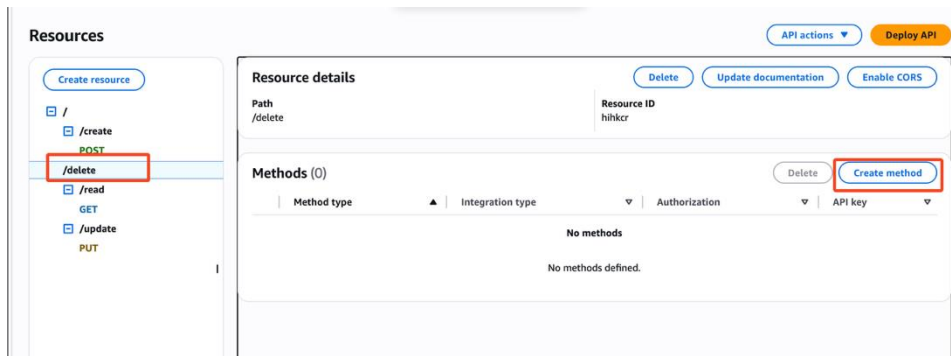
► URL query string parameters

► HTTP request headers

► Request body

Cancel Create method

DELETE - DELETE



Create method

Method details

Method type

DELETE

Integration type

☒ **Lambda function**
Integrate your API with a Lambda function.

☐ **AWS service**
Integrate with an AWS Service.

☐ **HTTP**
Integrate with an HTTP endpoint.

☐ **VPC link**
Integrate with a VPC link.

☒ **Lambda proxy integration**
Send the request to your Lambda function as a structured event.

Lambda function

Provide the Lambda function name or alias. You can also provide an ARN from another account.

us-east-1

☒ **Grant API Gateway permission to invoke your Lambda function**
When you save your changes, API Gateway updates your Lambda function's resource-based policy to allow this API to invoke it.

Integration timeout [Info](#)
By default, you can enter an integration timeout of 50 - 29,000 milliseconds. You can use Service Quotas to raise the integration timeout to greater than 29,000 ms.

29000

Method request settings

URL query string parameters

HTTP request headers

Request body

[Cancel](#) [Create method](#)

Deploy API

Resources

Create resource

- /
- /create
- POST
- /delete
- DELETE
- /read
- GET
- /update
- PUT

/delete - DELETE - Method execution

ARN: `arn:aws:execute-api:us-east-1:1672941607001:pl4pfeimz1/*/*/DELETE/delete` Resource ID: `hihkr`

Update documentation Delete

API actions **Deploy API**

Client → Method request → Integration request → Lambda integration

← Method response ← Integration response Proxy integration ←

***New stage* API** ✕

Create or select a stage where your API will be deployed. You can use the deployment history to revert or change the active deployment for a stage. [Learn more](#)

Stage

Choose an option ▲

- *New stage***
- No stage

Cancel Deploy

Deploy API ✕

Create or select a stage where your API will be deployed. You can use the deployment history to revert or change the active deployment for a stage. [Learn more](#)

Stage

New stage ▼

Stage name

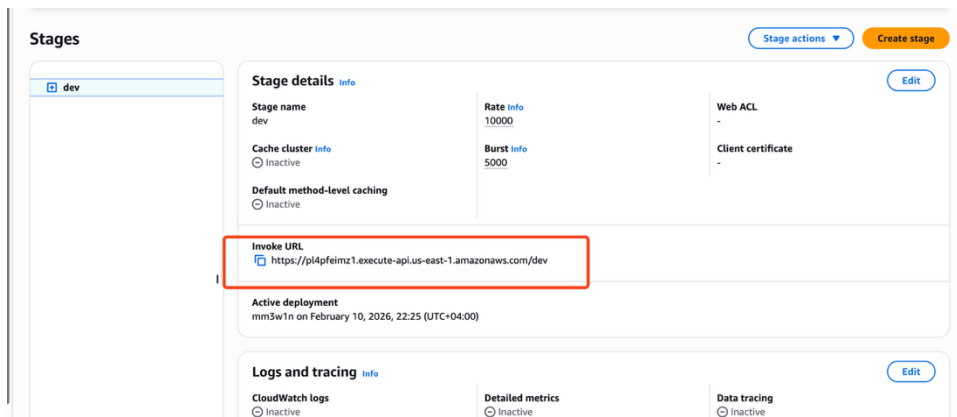
dev

A new stage will be created with the default settings. Edit your stage settings on the **Stage page.**

Deployment description

Cancel **Deploy**

Now we got the invoke url



<https://pl4pfemz1.execute-api.us-east-1.amazonaws.com/dev>

5.Final Testing

Open Postman and start doing the below

1. Creating the employee records

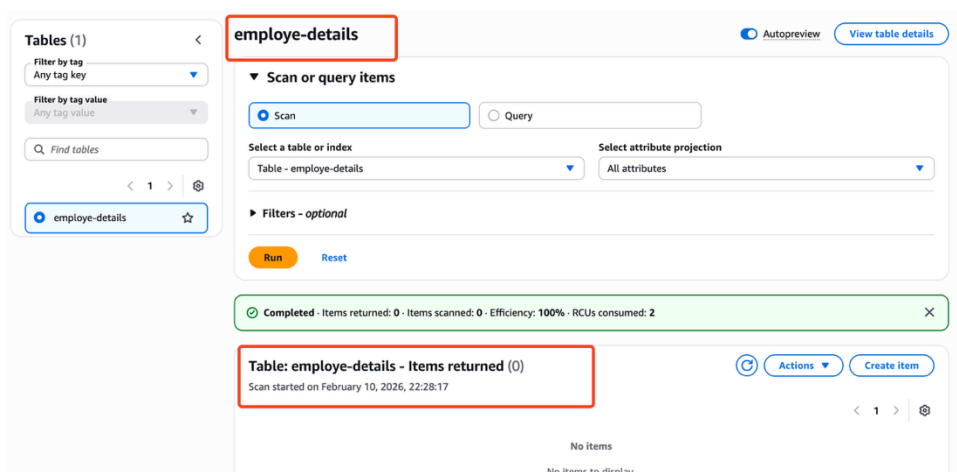
Method: POST

Resource: create

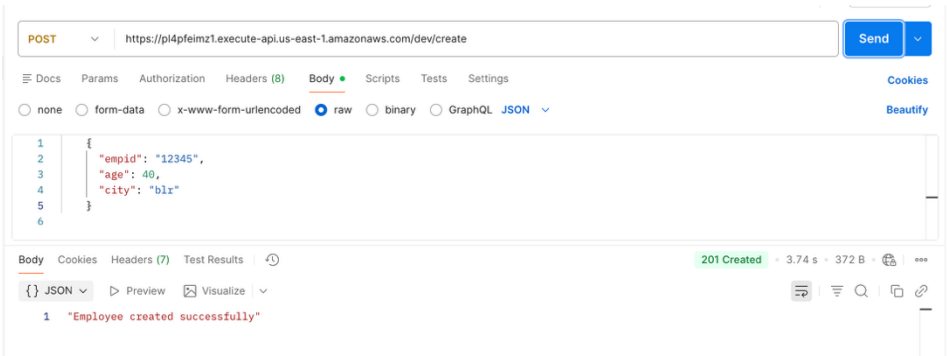
Body:

```
{
  "empid": "12345",
  "age": 40,
  "city": "blr"
}
```

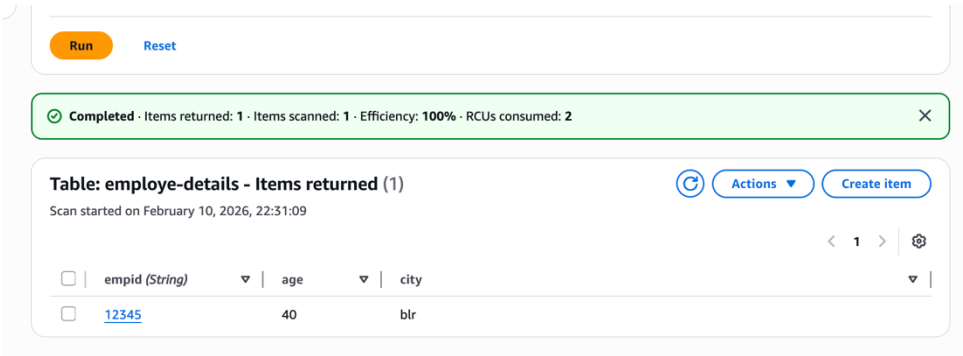
Before:



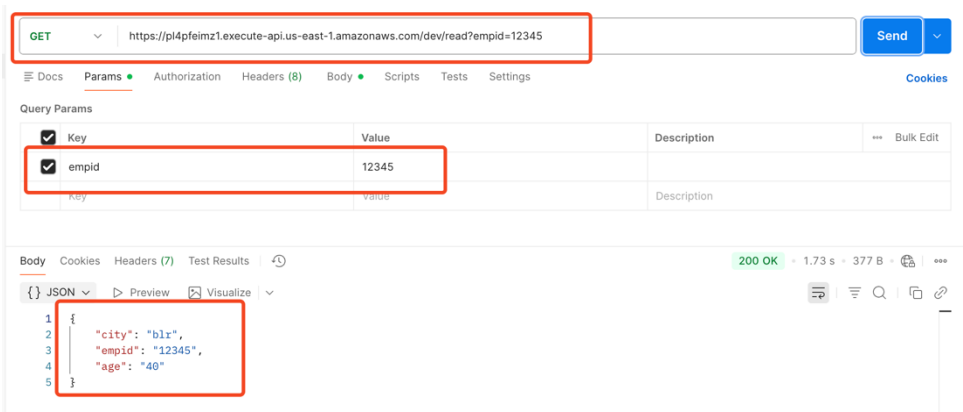
Postman:



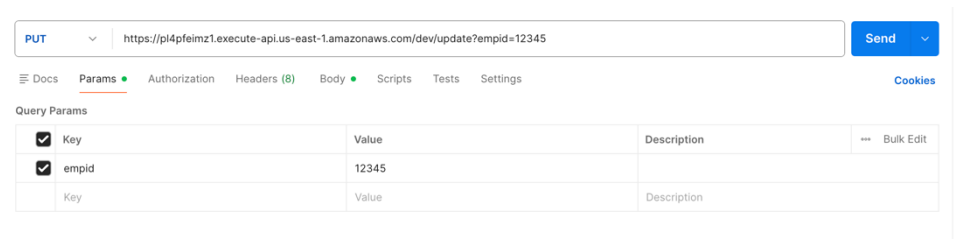
After:



2. Get employee records



3. Put employee records



DocsParamsAuthorizationHeaders (8)BodyScriptsTestsSettings

noneform-datax-www-form-urlencodedrawbinaryGraphQLJSON

1234

```
{
  "age": 50
}
```

PUThttps://pl4pfeimz1.execute-api.us-east-1.amazonaws.com/dev/update?empid=12345Send

DocsParamsAuthorizationHeaders (8)BodyScriptsTestsSettingsCookiesBeautify

1234

```
{
  "age": 50
}
```

BodyCookiesHeaders (7)Test Results

200 OK493 ms367 B

JSONPreviewVisualize

1"Employee updated successfully"

Table: employee-details - Items returned (1)

Scan started on February 10, 2026, 22:34:18

☐	empid (String)	age	city
☐	12345	50	blr

4. Delete employee records

DELETEhttps://pl4pfeimz1.execute-api.us-east-1.amazonaws.com/dev/delete?empid=12345Send

DocsParamsAuthorizationHeaders (6)BodyScriptsTestsSettingsCookies

Query Params

☒ Key	Value	Description
☒ empid	12345	
☐ Key	Value	Description

BodyCookiesHeaders (7)Test Results

200 OK2.64 s373 B

JSONPreviewVisualize

1"Employee 12345 deleted successfully"

6.Logging & Troubleshooting:

BUSINESS DOCUMENT This document is intended for business use and should be distributed to intended recipients only.

aws

cloudwatch

Ask Amazon Q

CloudWatch

Services

Features

Documentation

CloudWatch

Monitor Resources and Applications

Infrastructure Monitoring

▼ Logs

Log Management [New](#)

Log Anomalies

Live Tail

Logs Insights

Contributor Insights

▼ Metrics

All metrics

Set alarms on any of your resources to receive notification when they cross your specified thresholds.
[Create alarms](#)

Get started with Observability

CloudWatch observability tools help you monitor and running quickly with new services.

CloudWatch

Favorites and recents

Trace Map

Application Insights

▼ Infrastructure Monitoring

▼ Logs

Log Management [New](#)

Log Anomalies

Live Tail

Logs Insights

Contributor Insights

▼ Metrics

All metrics

Explorer

Streams

▼ Network Monitoring

Log groups

Data sources - new

Summary

► Get-set-go with unified data store

Log groups (7)

By default, we only load up to 10,000 log groups.

Filter log groups or try pattern search

☒ Exact match

☐	Log group	Log class	Anomaly d...	
☐	/aws/apigateway/welcome	Standard	Configure	
☐	/aws/lambda/AWS-PROJECT-LAMBDA	Standard	Configure	
☐	/aws/lambda/FINAL-TEST	Standard	Configure	
☐	/aws/lambda/dvs-api-gateway-lambda	Standard	Configure	
☐	/aws/lambda/lambdaapiauthorizer	Standard	Configure	
☐	API-Gateway-Execution-Logs_ab74kdmvoi/dev	Standard	Configure	
☐	API-Gateway-Execution-Logs_lqvl44xbyl/dev	Standard	Configure	

CloudShall Feedback Console Mobile App © 2026 Amazon Web Services, Inc. or its affiliates

2026-02-10T18:35:13.477Z END RequestId: 625efc42-5865-4053-8e53-c539053c81c2